



RAPPORT FINAL

Générateur de grilles de Sudoku à difficulté contrôlée

Groupe X

KONCHEV Martin

DOLO Abdourahamane-Abdoul

BARRE-GUEROT Ilan

BRIOT-AMADIUE Kellyan

Encadrant : Marc Hartley

L3 Informatique

Faculté des Sciences

Université de Montpellier

Avril 2026

1 Présentation du Sujet

1.1 Contexte et problématique

Le projet s'inscrit dans le domaine de la conception d'outils algorithmiques appliqués aux jeux logiques, et plus particulièrement au Sudoku. Ce jeu, largement répandu, consiste à remplir une grille 9×9 en respectant des contraintes strictes : chaque chiffre doit apparaître une seule fois par ligne, par colonne et par sous-grille 3×3 .

La problématique principale étudiée dans ce projet est la **génération automatique de grilles de Sudoku avec une difficulté contrôlée**, tout en garantissant qu'elles possèdent une solution unique. Ce problème est non trivial, car il implique plusieurs défis combinés : générer des grilles valides, contrôler leur complexité, et être capable de les résoudre efficacement pour en analyser la difficulté.

Dans le contexte du projet, l'objectif est de concevoir un système complet intégrant :

- un générateur de grilles,
- un solveur automatique,
- un mécanisme d'évaluation de la difficulté,
- ainsi qu'une interface permettant de manipuler les grilles.

Comme mentionné dans le rapport de Sprint 1, le projet vise à créer un outil capable de produire, analyser et manipuler des grilles de Sudoku de manière automatisée.

1.2 Intérêt du sujet

Ce projet présente un intérêt à la fois pédagogique et technique.

D'un point de vue académique, il mobilise plusieurs notions fondamentales de l'informatique :

- les algorithmes de recherche (notamment le backtracking),
- la gestion de structures de données (matrices représentant les grilles),
- l'analyse de complexité,
- et la conception logicielle modulaire.

Il s'inscrit pleinement dans le parcours de Licence Informatique, en permettant d'appliquer concrètement des concepts théoriques étudiés en cours.

D'un point de vue plus large, ce type de problème est représentatif de nombreuses situations en informatique, notamment dans les domaines de :

- l'intelligence artificielle (résolution de contraintes),
- la génération procédurale (création automatique de contenu),
- et les systèmes d'aide à la décision.

De plus, le contrôle de la difficulté d'un problème algorithmique est un enjeu important dans de nombreux domaines, allant du jeu vidéo à l'apprentissage adaptatif.

1.3 Approches possibles et choix retenu

Plusieurs approches peuvent être envisagées pour résoudre le problème de génération et de résolution de Sudoku :

1.3.1 Approches basées sur des techniques humaines

Ces méthodes reproduisent les stratégies utilisées par les joueurs humains (singletons, paires, etc.). Elles permettent de qualifier plus finement la difficulté d'une grille, mais sont souvent complexes à implémenter.

1.3.2 Approches algorithmiques classiques

Elles reposent sur des techniques comme :

- le backtracking,
- la recherche exhaustive avec contraintes,
- ou des heuristiques d'optimisation.

Ces méthodes sont robustes et garantissent la résolution complète des grilles.

1.3.3 Approches hybrides

Elles combinent les deux précédentes : un solveur automatique avec une analyse basée sur des techniques humaines pour évaluer la difficulté.

Choix retenu

Dans le cadre du Sprint 1, le choix s'est porté sur une approche principalement algorithmique, basée sur un solveur utilisant le backtracking.

Ce choix est justifié par :

- sa simplicité d'implémentation,
- sa fiabilité pour garantir une solution,
- et sa capacité à fournir des métriques de difficulté via le nombre de backtracks.

Comme précisé dans le rapport, la difficulté d'une grille est estimée à partir du nombre de retours arrière (backtracks) effectués lors de la résolution.

À terme, cette approche pourra être enrichie par des techniques de résolution humaines, prévues dans les sprints suivants.

1.4 Cahier des charges détaillé

Le cahier des charges du projet définit les fonctionnalités et contraintes principales du système.

1.4.1 Fonctionnalités principales

Le système doit permettre :

- la génération de grilles complètes valides,
- la création de grilles jouables avec solution unique,
- l'évaluation de la difficulté en fonction du processus de résolution,
- la résolution automatique des grilles,
- la lecture et l'écriture de grilles via un parseur de fichiers,
- ainsi qu'une interface console interactive permettant de jouer.

Le rapport précise que l'interface permet déjà :

- d'afficher une grille,
- de choisir une difficulté,
- d'insérer des valeurs,
- de voir les candidats possibles,
- et de vérifier leur validité.

1.4.2 Contraintes techniques

Le projet doit respecter plusieurs contraintes :

- développement en **Python**,
- utilisation de bibliothèques standards,
- exécution sur les environnements Linux de l'université,
- temps de calcul raisonnable pour la génération et la résolution,
- organisation du code via un dépôt Git structuré [4].

2 Technologies utilisées

2.1 Langages et outils utilisés

Dans le cadre de ce projet, plusieurs technologies et outils ont été mobilisés afin de concevoir, développer et organiser efficacement le système de génération et de résolution de grilles de Sudoku.

2.1.1 Langage de programmation

Le projet a été développé principalement en **Python**, qui constitue le cœur de l'implémentation. Ce langage est utilisé pour l'ensemble des composants du système :

- génération des grilles,
- résolution (solveur),
- évaluation de la difficulté,
- gestion des fichiers (parseur),
- interface utilisateur en console.

Comme indiqué dans le rapport de Sprint 1, Python a été choisi pour développer l'ensemble des modules du projet, en s'appuyant principalement sur des bibliothèques standards.

2.1.2 Bibliothèques utilisées

Le projet utilise essentiellement des bibliothèques natives de Python [2], notamment :

- **random** : pour la génération aléatoire des grilles,
- **os** : pour la gestion des fichiers et du système,
- **shutil** : pour certaines opérations sur les fichiers,
- **pickle** : permet de sauvegarder directement des objets Python dans un fichier,
- **matplotlib** : pour la visualisation des statistiques du solveur,
- **selenium** [6] : afin de récupérer une estimation de la difficulté d'une grille depuis le site Sudoku Coach [1],
- **webdriver_manager** : avec comme utilité d'adapter la récupération de difficulté aux différents navigateur web,
- **flask** : pour pouvoir interagir avec une interface plus pratique et intuitive [3],
- **flask_socketio** : permettant au serveur de communiquer avec le script javascript de la page web,
- **pytest** : afin de tester le bon fonctionnement des classes.

L'utilisation de bibliothèques standards permet de garantir la simplicité, la portabilité et la reproductibilité du projet.

2.1.3 Outils de développement

Gestion de version Le projet est géré à l'aide de **Git** [4], avec un dépôt hébergé sur GitLab. Cela permet :

- de suivre l'évolution du code,
- de travailler en équipe de manière collaborative,
- de gérer différentes versions via des branches.

Le rapport précise l'adoption d'un workflow simple avec une branche principale et des branches de développement.

Outils de communication La communication au sein du groupe est assurée via :

- Discord, pour les échanges rapides,
- des réunions hebdomadaires, permettant de suivre l'avancement et répartir les tâches.

Environnement d'exécution Le projet est conçu pour être exécuté sur :

- les machines Linux de l'université,
- ainsi que les ordinateurs personnels des membres du groupe.

Cette contrainte impose une attention particulière à la portabilité du code.

2.2 Justification des choix technologiques

2.2.1 Choix du langage Python

Le choix de Python s'explique par plusieurs raisons majeures.

Tout d'abord, Python est un langage simple et lisible, ce qui facilite la compréhension du code, notamment dans un contexte de travail en groupe. Cette lisibilité est particulièrement importante pour un projet impliquant plusieurs modules (générateur, solveur, interface, parseur).

Ensuite, Python est particulièrement adapté à l'implémentation d'algorithmes complexes, comme le backtracking utilisé pour résoudre les grilles de Sudoku. Sa syntaxe permet de se concentrer sur la logique algorithmique sans être alourdi par des contraintes techniques.

De plus, Python offre une grande flexibilité dans la manipulation des structures de données, comme les matrices utilisées pour représenter les grilles. Cela facilite la mise en place rapide de prototypes fonctionnels, comme celui réalisé lors du Sprint 1.

Enfin, Python est largement utilisé dans le domaine académique, ce qui en fait un choix cohérent avec le cadre du projet.

3 Développements Logiciel : Conception, Modélisation, Implémentation

3.1 Présentation des développements réalisés et modules principaux du logiciel

3.1.1 Grille (Figure 2)

Nous avons décidé de découper et représenter la grille en 3 parties, ces parties étant les cellules contenant les valeurs, les blocs qui correspondent aux carrés regroupant les cellules et que nous verrons par la suite sous le nom de 'zone', et enfin la grille en elle-même qui est le carré regroupant les zones (**Figure 3**). Grâce à cette représentation, nous avons ensuite aisément pu implémenter le concept de grille avec une classe pour chacune des parties énumérées ci-dessus.

Tout d'abord, la classe cellule contient évidemment un attribut pour sa valeur, mais aussi un attribut pour les candidats de la cellule sous forme d'une liste d'entiers, sachant qu'un candidat est une valeur qui peut être attribuée à la cellule sans rompre les règles fondamentales du sudoku, mais qui n'est pas forcément la bonne. Par "la bonne" il est entendu que cette valeur est la seule valeur possible pour cette cellule permettant au sudoku d'être résolu, qui nous le rappelons a pour principe de n'avoir qu'une seule solution possible.

Pour ce qui est des méthodes de cette classe, il y a bien évidemment des accesseurs et des mutateurs pour les attributs cités précédemment mais aussi pour un troisième attribut nommé position qui fut rajouté par la suite pour faciliter et alléger la résolution de grilles.

Ensuite nous avons la classe correspondant aux zones et qui, étant donné qu'une zone est un regroupement de cellule, a un attribut contenant ces dites cellules à l'aide d'une liste de cellules ainsi que l'attribut représentant la taille d'un côté de tel manière à ce que la taille au carré correspond au nombre de cellules dans la zone.

Cependant, contrairement aux cellules, il n'a été établi qu'un accesseur pour la taille pour limiter ainsi que sécuriser les accès. Pour ce qui est du reste, nous avons principalement des méthodes renvoyant les valeurs des cellules sous différentes formes ainsi que des accesseurs à une cellule identifiée soit par sa position ou ses coordonnées (ligne et colonne) dans la zone. Cette manière d'indexer les cellules, comme vous pourrez le constater, sera très redondante tout au long de la manipulation des cellules, avec des avantages et des inconvénients pour chacune.

Enfin, la grille est la classe la plus importante et la plus grande, elle donne un accès rapide et pratique aux cellules contenu dans ces zones par le biais des méthodes de la classe zone et avec l'aide des fonctions principalement arithmétiques contenues dans le fichier GrilleUtils.py dont le but est de faire des calculs d'index. De ce fait, ses méthodes ne seront pas présentées en détail. Similairement à la classe des zones, la grille contient une liste de zones, une taille avec laquelle on peut obtenir le nombre de zones lorsqu'on la met au carré mais aussi sa difficulté sous la forme d'une énumération.

Quand aux méthodes, nous avons premièrement un accesseur et mutateur pour la difficulté, un accesseur pour la taille et des accesseurs et mutateurs pour les cellules. Cependant, pour encore une fois garder une certaine confidentialité et sécurité, les méthodes concernant les cellules sont privées et servent de fondations aux multiples méthodes permettant d'accéder et manipuler les valeurs et candidats des cellules.

Une méthode renvoyant les régions fût ensuite rajoutée, avec une région correspondant soit à une zone, ligne ou colonne. En effet nous avons choisi de regrouper les cellules en zone parce qu'elles sont plus intuitives et visuelles, mais il est aussi possible de les remplacer par les lignes ou colonnes de cellules, nous avons donc décidé de renvoyer ces différents formats afin d'éviter les calculs d'index interminables dans la classe de résolution de grilles (Solver) et centraliser ces mêmes calculs au sein de la classe Grille.

Dernièrement, la grille contient une génération de valeurs sous forme de méthode abstraite étant donné qu'elle dépendra des différentes approches utilisés.

Pour finir et comme énoncé précédemment, la génération des valeurs n'est pas définie dans la classe Grille, qui devient donc une classe abstraite, et la génération est relayée à ses classes filles, chacune correspondant à une différente manière de générer les valeurs de la grille de sudoku.

3.1.2 Solver (Figure 4)

Contexte et objectif : Le projet s'organise autour d'une hiérarchie de classes dont la racine abstraite est Solver. Deux familles de solveurs en héritent : SolverBacktrack, qui implémente une résolution exhaustive par retour sur trace (backtracking), et SolverHuman, qui simule les raisonnements humains classiques à travers une succession de techniques de complexité croissante. Chaque famille dispose d'une variante instrumentée (SolverBacktrackStats et SolverHumanStats) permettant de collecter des métriques de résolution.

Architecture générale : L'organisation du code repose sur le principe de l'héritage. La figure 1 en donne une vue synthétique.

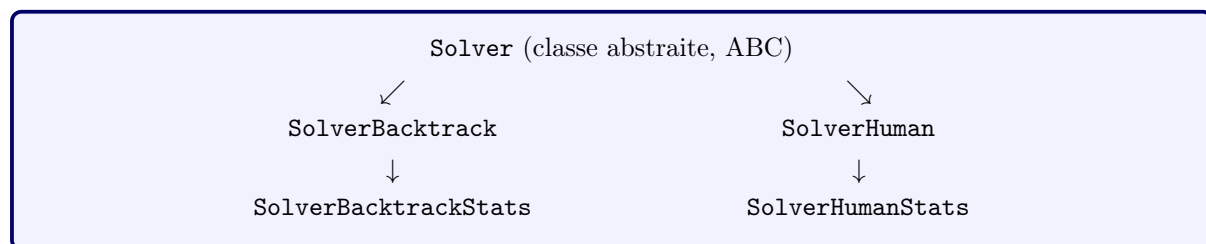


FIGURE 1 – Hiérarchie des classes solveurs

Objet Grille (structure d'entrée commune) : Toutes les classes de la hiérarchie reçoivent en entrée un unique paramètre : un objet de type Grille. Cet objet encapsule l'intégralité de l'état courant de la grille de Sudoku et expose une interface uniforme utilisée par tous les solveurs.

Les dimensions dérivées s'obtiennent ainsi :

$$n = \text{getSize}(), \quad N = n^2 \text{ (taille d'une ligne/colonne),} \quad \text{nb cellules} = N^2$$

Pour un Sudoku 9×9 standard : $n = 3$, $N = 9$, 81 cellules.

Classe mère (Solver) : est une classe abstraite (ABC) qui définit le contrat commun à toutes les stratégies de résolution. Elle ne possède aucun attribut d'instance et ne stocke aucun état.

Méthode abstraite : Toute sous-classe doit implémenter `solveGrille(grille: Grille) -> bool`.

Paramètre	Description
<code>grille</code>	Objet Grille en entrée/sortie — modifié en place par le solveur
Retour	<code>True</code> si la grille est résolue, <code>False</code> sinon

Méthode de validation (`isValid`) :

Cette méthode statique primordiale est partagée dans toute la hiérarchie. Elle vérifie qu'une valeur candidate ne viole aucune des trois contraintes du Sudoku (colonne, ligne, bloc).

La classe `SolverBacktrack` : ne définit aucun attribut propre : toute l'information réside dans l'objet Grille passé en paramètre. Le parcours s'effectue en coordonnées (i, j) avec $i, j \in [0, N - 1]$, ligne par ligne de gauche à droite.

Génération des valeurs candidates : les valeurs 1 à N sont générées sous forme de liste Python, puis mélangées aléatoirement via `random.shuffle` dans `solveGrille`. Ce choix garantit la diversité des solutions produites lors de la génération de grilles.

Classe `SolverBacktrackStats` : hérite de `SolverBacktrack` et ajoute un dictionnaire de classe partagé entre toutes les instances, avec les clés suivantes :

Clé	Type	Description
<code>appelsRecurifs</code>	<code>int</code>	Profondeur totale de l'arbre de recherche
<code>testsEffectues</code>	<code>int</code>	Appels à <code>isValid</code>
<code>nbBacktracks</code>	<code>int</code>	Remises à zéro après échec
<code>valid</code>	<code>bool</code>	Succès de la résolution

Les compteurs sont incrémentés à des points précis de l'algorithme :

1. `appelsRecurifs` += 1 à l'entrée de chaque appel récursif ;
2. `testsEffectues` += 1 avant chaque appel à `isValid` ;
3. `nbBacktracks` += 1 après chaque remise à zéro d'une cellule ;
4. `valid` est mis à `True` dès qu'une solution complète est trouvée.

Classe `SolverHuman`

`SolverHuman` (résolution par techniques humaines) : Contrairement au backtracking, `SolverHuman` exploite les candidats de chaque cellule. Avant tout traitement, la méthode `grille.adjustCandidates()` est appelée pour initialiser ces listes : pour chaque cellule vide, elle calcule l'ensemble des valeurs $\{1, \dots, N\}$ non encore présentes dans sa ligne, sa colonne, et son bloc.

Principales structures accédées à l'exécution :

- Régions (`getRegions()`) — liste cyclique (zone / ligne / colonne) ;
- Candidats — liste d'entiers par cellule, modifiée en place pendant la résolution ;
- Valeurs — entier $\in \{0, 1, \dots, N\}$ par cellule (0 = vide).

`solveGrille` orchestre les techniques dans un ordre de complexité croissante. Si une technique réussit, la boucle repart depuis le début (`continue`). Si aucune ne progresse, une `SolverError` est levée.

Ordre	Technique	Type	Action
1	dernierNombre	Attribution	Place la valeur manquante
2	singletonNu	Attribution	Candidat unique dans une cellule
3	singletonCache	Attribution	Candidat unique dans une région
4	paireNu	Élimination	Paire de candidats verrouillée
5	paireCachee	Élimination	Paire de valeurs verrouillée
6	candidatEnferme	Élimination	Candidat confiné à une intersection
7	gratteCiel	Élimination	Chaîne à deux sommets

SolverHumanStats (variante instrumentée) : contrairement à SolverBacktrackStats, les statistiques sont ici des variables locales initialisées à chaque appel de `solveWithStats`, ce qui évite les problèmes de persistance entre résolutions. Chaque entrée de l'historique ("history") est un dictionnaire contenant le numéro d'étape, la technique utilisée, ainsi que la ligne, la colonne et la valeur affectée.

Le score de difficulté est calculé par `scoreFromStats` / `rateFromStats` selon le tableau suivant :

Score	Label	Technique déclenchante
≤ 1.6	Easy	Dernier nombre / Singleton nu
≤ 2.6	Medium	Singleton caché
≤ 4.2	Hard	Paire nue / Paire cachée
≤ 6.4	Vicious	Candidat enfermé / Gratte-ciel
> 6.4	Fiendish+ / God Mode	Grille non soluble par les techniques

Le score de base est donné par la technique maximale utilisée, puis légèrement affiné par les compteurs d'usage de chaque technique (contribution plafonnée pour éviter qu'un volume important d'une technique facile ne dépasse la classe d'une technique difficile).

3.1.3 Parser (Figure 5)

Le Parser est une classe faite en 2 parties comme nous pouvons le voir dans le Diagramme. La majorité de la Classe est statique, mais une partie nécessite l'instanciation.

1. Le Parser :

La classe Parser (Fichier Parser.py) a comme objectif de faire du transtypage. Dans notre projet, nous avons comme ambition de pouvoir faire de l'enregistrement de partie, afin de pouvoir sauvegarder une progression par exemple. Mais également l'idée de stocker le déroulement d'une partie ou bien tout le processus de résolution. Également, dans le Scrapper par exemple nous avons besoin d'avoir notre Grille sous forme textuelle, c'est donc ici que le Parser à vue une raison de naître. Il peut donc faire tout un tas de conversion, nous pouvons le voir dans notre Diagramme UML (**Figure 5**). Des exemples sont `grilleToTab`, `tabToGrille`, `grilleToStringWithoutCandidates`, il y a donc, évidemment, des moyens de faire les transtypages dans le deux sens, `grilleToTab` transforme notre Grille en tableau à deux dimensions, et `tabToGrille` fait l'inverse. Sachant que les candidats, au début du projet n'étaient pas encore bien mis en place. La classe Parser, prend dans la majorité de ses méthodes pas en compte les candidats, les méthodes citées précédemment en font partie (`grilleToTab...`). Comme dit précédemment, c'est une classe en grande partie statique, en réalité tout ce qui ne touche pas à l'écriture et la lecture de fichier est statique, ce qui est cohérent, car il n'y a besoin de rien pour faire du simple transtypage. Cependant, lorsque nous voulons faire de l'écriture/lecture dans un fichier, on a deux méthodes : `grilleToFile` et `fileToGrille` ; qui respectivement permette la lecture et l'écriture d'une grille dans un fichier. Ici, l'instanciation sera nécessaire, car nous utilisons une classe `FileInteraction` qui va être décrite ci-dessous.

2. FileInteraction

La classe `FileInteraction` a été créée afin de faciliter l'écriture et la lecture dans un même répertoire ou dans un même fichier tout en minimisant le temps d'ouverture des fichiers. On y trouvera des méthodes

pour choisir l'emplacement de lecture et d'écriture (`setDirectory`, `setFile...`) et des méthodes d'écriture et de lecture (`read`, `write...`). Ici, les méthodes de lecture et d'écriture ne revoient pas de manière brute le texte lu, on renvoie un tableau de chaque ligne afin de faciliter le traitement notamment dans `Parser`.

3.1.4 Scraper

Dans un premier temps, la classe `SudokuScraping` a été implémentée afin de comparer la difficulté estimée par notre algorithme avec celle fournie par le site `sudoku.coach`, qui dispose d'une métrique reconnue. La partie web scraping repose sur des structures simples. La grille en entrée est représentée sous forme d'une chaîne de 81 caractères, où chaque chiffre correspond à une cellule (0 représentant une case vide), par exemple : "204503910060008005...".

Le résultat retourné par `get_difficulty_form_coach()` est un dictionnaire Python de la forme suivante : "score" : 2, "label" : "Modérément facile" où score est un entier compris entre 0 et 10 et label correspond au texte affiché par `sudoku.coach`.

La fonction principale `get_difficulty_form_coach()` attend une chaîne composée exactement de 81 caractères numériques. Cette convention correspond au format standard de représentation d'une grille de sudoku 9×9 sous forme linéaire, lue de gauche à droite puis de haut en bas. La fonction `Parser.grilleToStringWithoutCandidates()` se charge de convertir une instance de `GrilleHuman` vers ce format en parcourant les cellules via `getCellValueCoord(row, col)`.

La récupération des données se fait en deux étapes. Dans un premier temps, Selenium lance Firefox en mode headless et charge l'URL : `https://sudoku.coach/fr/solver/`. Ensuite, après avoir attendu la fin du calcul du solveur JavaScript de `sudoku.coach` (environ 6 secondes), le texte brut de la page est extrait à l'aide de `driver.find_element(By.TAG_NAME, "body").text`.

La validation du résultat est effectuée à l'aide d'une expression régulière dans la méthode `_find_difficulty()` : `re.search(r'Difficulté\s*:\s*\n([\n]+)\n(\d+)', page_text)`

Ce motif recherche précisément le bloc affiché par `sudoku.coach`, de la forme « Difficulté : Modérément facile (2) ».

Si le motif n'est pas trouvé (page non chargée, site indisponible ou modification du format), la fonction retourne (`None`, `None`) sans lever d'exception, permettant ainsi au programme de continuer son exécution.

3.2 Diagramme de cas d'utilisation (UML)

Voici le diagramme de cas d'utilisation UML : **Figure 7**

3.3 Diagramme de classes (UML)

Voici une version simplifiée du diagramme de classes UML : **Figure 8**

3.4 Fonctionnalités de l'interface (console) (Figure 6)

Le projet propose une interface console interactive permettant à l'utilisateur d'interagir avec le système de génération et de résolution de Sudoku. Cette interface joue un rôle central dans le projet, car elle permet de tester les fonctionnalités développées tout en offrant une expérience utilisateur simple et fonctionnelle.

Au lancement du programme, un menu principal est affiché. L'utilisateur peut alors choisir entre générer une grille de Sudoku et jouer, ou quitter l'application. Ce point d'entrée est volontairement simple afin de rendre l'utilisation du programme intuitive.

Une fois le jeu lancé, l'utilisateur est invité à choisir un niveau de difficulté parmi plusieurs options : facile, moyen, difficile, extrême et un mode avancé ("God Mode"). Ce choix influence directement la génération de la grille, notamment le nombre de cases vides et la complexité de résolution.

Après la génération, la grille est affichée dans la console. L'interface fournit également des informations supplémentaires sur la difficulté, telles que :

- une estimation basée sur des méthodes de résolution humaines,
- le nombre d'étapes nécessaires pour résoudre la grille,
- la technique la plus complexe utilisée.

Ces informations permettent à l'utilisateur de mieux comprendre le niveau réel de la grille.

L'utilisateur peut ensuite interagir avec la grille via plusieurs actions. Il peut notamment entrer une valeur dans une case, en précisant la ligne, la colonne et la valeur souhaitée. Le système vérifie alors que la saisie est valide : les coordonnées doivent être correctes, la case ne doit pas être déjà remplie, et la valeur doit respecter les règles du Sudoku. Si une erreur est détectée, un message explicite est affiché.

Un système de gestion des erreurs est également mis en place. L'utilisateur dispose d'un maximum de trois erreurs ; au-delà de cette limite, la partie se termine automatiquement avec un message de type « Game Over ». Cela ajoute une dimension de contrainte et rend l'interaction plus dynamique.

L'interface propose également d'afficher les candidats possibles pour chaque case vide. Cette fonctionnalité permet de visualiser toutes les valeurs envisageables dans une case donnée, ce qui aide à comprendre le processus de résolution et peut servir d'outil pédagogique.

En complément, l'utilisateur peut choisir de résoudre automatiquement la grille selon deux approches différentes :

- une résolution complète par backtracking (méthode algorithmique),
- une résolution basée sur des techniques humaines, accompagnée de statistiques (nombre d'étapes, techniques utilisées, etc.).

Ces deux méthodes permettent de comparer les approches et d'analyser la difficulté de la grille générée.

3.4.1 Exemples d'affichage dans l'interface console avec la résolution humaine

Comme illustré en **Figure 9**, l'interface console affiche la grille chargée ainsi que les informations liées à sa difficulté (facile).

Un exemple d'une génération de grille difficile est présenté en **Figure 10**.

Après la génération ou le chargement d'une grille, l'utilisateur peut choisir d'afficher la résolution par méthodes humaines afin de terminer la partie sans saisir manuellement les valeurs. Le programme montre alors les différentes étapes appliquées par le solveur humain. La **Figure 11** présente un exemple de résolution d'une grille extrême.

Dans le cas de la résolution par méthode humaine, le programme peut également afficher les étapes détaillées de résolution avant la fin de la partie. Chaque étape indique la technique utilisée, la valeur placée, ainsi que sa position dans la grille **Figure 12**. Cela permet de suivre le raisonnement du solveur humain et de mieux comprendre pourquoi une grille est classée dans une certaine difficulté avant le retour au menu principal.

Le chemin d'exécution est celui-ci : Le chemin d'exécution a été modifié afin d'éviter la génération directe des grilles pendant l'utilisation normale du programme. Désormais, lorsqu'une partie est lancée, le programme ne génère plus une nouvelle grille en temps réel. Il charge une grille déjà préparée depuis la banque de grilles **Figure 13**.

Le nouveau chemin d'exécution est le suivant :

```
InterfaceConsole.startPlaying() → playSudoku() → askDifficulty() →  
BanqueGrilles.charger_grille_aleatoire(difficulty) → BanqueGrilles.charger_banque() →  
pickle.load() → random.choice() → gameLoop()
```

La méthode `charger_banque()` lit le fichier `banque_grilles.pkl`, qui contient les grilles pré-générées. Ensuite, `charger_grille_aleatoire()` sélectionne au hasard une grille correspondant à la difficulté choisie par l'utilisateur. Cette grille contient déjà la grille de jeu, la solution complète et les statistiques de résolution calculées auparavant avec `SolverHumanStats`.

La génération complète existe encore, mais elle est maintenant utilisée uniquement avant la démonstration, dans le script de préparation :

```
PreparerBanque.py → GrilleBacktrack.generateEntireGrille() →  
GrilleHuman.generateValuesHumanRated() → SolverHumanStats.solveWithStats() →  
SolverHuman → BanqueGrilles.ajouter_grille() → BanqueGrilles.sauvegarder_banque() →  
pickle.dump()
```

Cette étape permet de générer plusieurs grilles pour chaque difficulté, puis de les sauvegarder dans le fichier `banque_grilles.pkl`.

Ainsi, même si elle reste simple, cette interface console offre un ensemble de fonctionnalités complet permettant de générer, manipuler, résoudre et analyser des grilles de Sudoku. Elle constitue une base solide pour une éventuelle évolution vers une interface graphique plus avancée.

3.5 Structures de données principales

Comme vous pouvez le constater tout au long de ce rapport, notre choix de structure s'est très rapidement tourné vers les classes du paradigme de Programmation Orientée Objet, principalement pour leur clarté, visualisation et efficacité quand il s'agit de communiquer entre différents modules. De plus, les énumérations se sont trouvées être un choix astucieux pour représenter une statistique avec un nombre fini de valeur possible, tel que la difficulté avec comme valeur soit facile, moyen, difficile, extrême ou encore godmode.

3.6 Format des entrées (grilles Sudoku)

- grille 9x9
- valeurs entre 1 et 9
- 0 pour représenter une case vide
- stockage en matrice ou dans des objets Cellule
- éventuellement lecture depuis un fichier
- grille complète (solution) / grille de jeu

Les grilles peuvent aussi être sauvegardées dans la banque de grilles pré-générées. Dans ce cas, chaque entrée contient la grille de jeu, la solution complète, la difficulté et les statistiques associées comme déjà mentionné plus bas dans le rapport (7.1.1).

3.7 Procédures de lecture et validation

- vérifier que les valeurs sont entre 0 et 9
- vérifier qu'une valeur entrée par l'utilisateur est valide
- vérifier ligne / colonne / bloc 3x3
- vérifier que la case n'est pas déjà remplie
- comparer avec la grille solution
- ajuster les candidats possibles

Après chaque insertion valide, les candidats des autres cellules sont mis à jour. Cela permet de conserver une grille cohérente et de faciliter l'affichage des candidats possibles.

- validation par solveur de backtrack ou solveur humain

3.8 Statistiques

Cette section présente quelques statistiques générales sur la taille du projet. Ces chiffres permettent d'avoir une vue d'ensemble de l'organisation du code et de la répartition des fichiers.

3.8.1 Complexité des fichiers principaux de l'interface console

Fichier	Rôle principal	Complexité estimée
InterfaceConsole.py	Gestion du menu, choix de difficulté, actions utilisateur, affichage de la grille, saisie des valeurs et lancement des résolutions automatiques(backtrack et humaine).	Moyenne à élevée
BanqueGrilles.py	Chargement et sauvegarde des grilles pré-générées avec <code>pickle</code> , choix aléatoire d'une grille selon la difficulté.	Faible à moyenne
GrilleHuman.py	Génération de grilles selon les techniques humaines, vérification de la difficulté cible et gestion des redémarrages pendant la génération.	Élevée
SolverHuman.py	Implémentation des techniques humaines de résolution : dernier nombre, singleton nu, singleton caché, paire nue, paire cachée, candidat enfermé, x-wing et gratte-ciel.	Élevée
SolverHumanStats.py	Résolution avec statistiques : nombre d'étapes, techniques utilisées, technique maximale et historique des coups.	Moyenne à élevée

TABLE 1 – Complexité des fichiers principaux utilisés par l'interface console

3.8.2 Nombre de modules / classes

Le projet contient environ **35 fichiers Python**. Chaque fichier Python peut être considéré comme un module, car il regroupe une partie précise du programme : gestion des grilles, génération, résolution, interface, historique, difficultés, techniques de résolution ou encore sauvegarde des grilles pré-générées.

Les fichiers Python n'ont pas tous la même taille. Certains sont assez courts, avec environ une vingtaine de lignes, tandis que d'autres sont beaucoup plus importants et peuvent dépasser les 300 lignes. En moyenne, les fichiers Python contiennent environ **150 à 200 lignes**.

Le projet contient aussi plusieurs fichiers liés à l'interface web et aux données :

Type de fichier	Nombre de fichiers
Python .py	35
HTML	1
CSS	2
JavaScript	1
JSON	2

TABLE 2 – Répartition approximative des fichiers du projet

Cette organisation montre que le projet est principalement développé en Python, avec quelques fichiers supplémentaires pour l'interface web et la configuration.

3.8.3 Nombre de lignes de code

En estimant le nombre de lignes à partir des fichiers du projet, on obtient environ :

- **Python** : 35 fichiers avec environ 150 à 200 lignes en moyenne, soit environ 5250 à 7000 lignes ;
- **HTML** : 1 fichier avec environ 440 ;
- **CSS** : 2 fichiers, avec environ 70 lignes pour l'un et 700 lignes pour l'autre, soit environ 770 lignes ;
- **JavaScript** : 1 fichier d'environ 8 lignes ;

— **JSON** : 2 fichiers contenant environ 1050 lignes au total.

On peut résumer cette estimation dans le tableau suivant :

Type de fichier	Nombre approximatif de lignes
Python	5250 à 7000
HTML	440
CSS	770
JavaScript	8
JSON	1050
Total	7138 à 8888

TABLE 3 – Estimation du nombre de lignes de code du projet

Le projet contient donc environ **7000 à 9000 lignes** au total. Cette estimation inclut le code, les commentaires, les lignes vides ainsi que les fichiers de données ou de configuration. Le nombre exact peut varier légèrement selon la méthode de comptage utilisée.

La majorité du code se trouve dans les fichiers Python, car ils contiennent toute la logique principale du projet : génération des grilles, résolution, classification par difficulté, gestion de l'historique, banque de grilles pré-générées et interaction avec l'utilisateur.

4 Algorithmes et Analyse

Tout d'abord, il est important de noter que les algorithmes sont tout au long de ce projet divisées en deux catégories :

- les algorithmes de backtracking,
- et les algorithmes employant des méthodes plus humaines.

Nous allons donc premièrement voir à quoi servent ces algorithmes et leur fonctionnement basique puis ensuite comparer leur complexité en temps.

4.1 Algorithmes principaux

4.1.1 Algorithmes de backtracking

Nous avons dans un premier temps un algorithme de génération de grilles par le biais du backtracking, qui consiste à générer une grille vide, la remplir, puis enlever un nombre de valeurs dépendant de la difficulté.

Le remplissage se fait tout simplement en appliquant l'algorithme de résolution de grille (nommé Solver) à la grille vide, tout en sachant que le Solver utilise des valeurs aléatoires, ce qui nous garanti une probabilité négligeable d'obtenir la même grille de départ.

Les valeurs à retirer quand à elles sont réparties en deux sous-algorithmes, un premier qui tente de retirer une valeur aléatoire jusqu'à ce que cela soit possible tout en conservant le principe de solution unique, et un deuxième utilisant le premier sous-algorithmes pour retirer des valeurs tant que c'est possible (et seulement jusqu'au nombre de retrait attendu bien évidemment) ou revenant sur ces pas si ce n'est pas possible, cette impossibilité signifiant que le choix de valeur précédent ne permet pas de créer une grille à solution unique sur le long terme.

Nous noterons que cette tactique de génération est fondamentalement la même pour la génération humaine et que la différence réside dans le Solver et dans l'évaluation des valeurs à retirer.

Ensuite l'algorithme de résolution de grille cité précédemment et dénommé Solver, nous nous sommes inspirés des différents cours d'algorithmique sur le sujet tel que le cours de recherche exhaustive et backtrack d'Algorithmique 4 [5], le concept est très simple : tester toutes solutions possibles jusqu'à trouver la bonne, tout en revenant sur ses pas si les grilles partant de la grille actuelle ne sont pas valides.

4.1.2 Algorithmes humains

Comme dit précédemment, la génération de grille humaine adopte la même tactique, avec seulement les sous-algorithmes qui diffère, la plus complexe différence étant le retrait de valeurs.

En effet, les valeurs ne sont plus retirées aléatoirement mais en fonction des techniques employables pour résoudre ce retrait, avec dépendamment de la difficulté une technique de résolution servant de plafond et pour laquelle les retraits demandant des techniques de complexité supérieure ne sont pas autorisés. Par complexité nous entendons à quel point la technique de résolution est avancé et difficile à exécuter pour l'utilisateur.

De plus, il n'y a maintenant non pas un nombre de retrait fixe mais un nombre de retrait minimum et maximum dépendant aussi de la difficulté.

Enfin, le Solver est maintenant certes plus compliqué mais la tactique n'en reste pas moins extrêmement simple, elle consiste juste à appliquer les techniques de la plus simple à la plus complexe jusqu'à ce qu'une soit applicable, et cela en boucle jusqu'à résoudre la grille. En effet, contrairement au Solver backtrack nous appliquons des techniques intelligentes garantissant soit l'ajout d'une valeur ou l'élimination de candidats, ce qui amènera à coup sûr à la résolution d'une grille valide.

4.2 Complexité temporelle des algorithmes

Afin d'avoir une complexité en temps sensée, nous allons d'abord établir n comme le nombre de retrait et m pour la taille d'un coté d'une zone de la grille (car c'est le seul indice que l'on peut incrémenter de un tout en gardant une grille carrée de sudoku), avec n le nombre de retrait maximum pour les algorithmes humains, et nous allons calculer la complexité des Solver avant celle de leur générateur respectif car les générateurs font appel au Solver.

4.2.1 Complexité backtrack

Nous avons donc en premier le Solver de backtracking avec la méthode `Solver.isValid` et un appel récursif avec une valeur ajoutée à la grille, tout deux contenus dans 3 boucles imbriquées itérant m^2 fois, cela nous donne

$$(m^2)^{3n}$$

, soit

$$O(m^{6n})$$

ce qui heureusement n'arrive jamais vu que l'algorithme est un algorithme de backtrack et non un algorithme exhaustif.

Le générateur quand à lui crée une grille vide sans faire recours à la difficulté donc $O(1)$, appel le solver pour le remplissage en $O(m^{6n})$, puis estime le nombre de retrait en fonction de la difficulté avec des calculs arithmétiques donc $O(1)$ aussi, et enfin effectue le retrait des valeurs.

Le retrait des valeurs ne fait simplement que boucler jusqu'à maximum $n \cdot m^2$ fois avec un retrait de valeur à l'intérieur qui boucle m^4 sur la vérification unicité qui revient à boucler m^6 fois sur un appel récursif avec une valeur ajoutée à grille. Cela nous donne $O(n \cdot m^6)$ pour la vérification d'unicité, $O(n \cdot m^4 \cdot m^6)$ qui est égal à $O(n \cdot m^{10})$ pour le retrait d'une valeur, $O(n \cdot m^2 \cdot n \cdot m^{10})$ soit $O(n^2 \cdot m^{12})$ pour le retrait des valeurs et pour finir $O(m^{6n}) + O(n^2 \cdot m^{12})$ qui donne $O(m^{6n})$ pour la génération de grille.

4.2.2 Complexité humaine

Pour ce qui est du Solver humain, nous allons d'abord déterminer qu'est-ce que le pire des cas. Dans le pire des cas, nous allons devoir utiliser la technique d'élimination de candidats avec la plus grande complexité jusqu'à éliminer tout les candidats (soit environ $m^4 \cdot m^2 = m^6$ candidats) puis la technique d'ajout de valeur avec la plus grande complexité jusqu'à remplir toutes les cellules libres (soit n fois).

Il se trouve que la technique d'ajout de plus haute complexité est `singletonCache` testant chaque valeur possible sur toutes les régions, soit m^2 valeurs fois $m^2 \cdot m^2 \cdot 3$, ce qui nous donne une complexité en $O(m^6)$.

Pour la technique d'élimination de candidats, on a `candidatEnferme` qui au pire parcourt toutes les régions ($m^4 \cdot 3$), pour lequel nous testons toutes les valeurs possibles (m^2), dans toutes les régions ($m^4 \cdot 3$), soit une complexité en $O(m^{10} \cdot 9)$.

Si nous assemblons toutes ses complexités, nous obtenons une complexité total de $O((m^{10} \cdot 9) \cdot m^6 \cdot m^6 \cdot n)$, soit $O(m^{22} \cdot n)$. Toutefois il est important de rappeler que non seulement les techniques d'élimination éliminent souvent plus d'un seul candidat et qu'il n'y a absolument pas besoin d'éliminer tout les candidats pour résoudre la grille, cette complexité est donc purement théorique et ne sera jamais atteinte.

5 Analyse des Résultats

5.1 Graphiques de performance

5.2 Analyse des résultats

5.3 Comparaison des méthodes

5.3.1 Backtracking vs Méthodes humaines

5.4 Procédures de test

5.4.1 Génération de grilles

5.4.2 Validation des solutions

6 Gestion du Projet

Au début du projet, soit le sprint1, nous nous sommes surtout concentrés sur l'aspect fonctionnel du projet, le but était d'avoir une génération de grille aléatoire, une résolution de grille et une interface affichant une grille avec laquelle on peut interagir.

Cependant, même si le projet était fonctionnel, il restait encore assez simple et pouvait être amélioré, qui nous laissa insatisfait après cette fin de sprint. Nous avons donc décidé de réorganiser le projet, de mettre des règles, conventions et répartitions de tâches ainsi que d'établir un diagramme des classes.

6.1 Organisation du projet

Premièrement, nous sommes passés à une structure avec des classes, ainsi que quelques énumérations, puis nous avons établi de multiples conventions tel que celles énumérées ci-dessous :

- avoir un fichier par classe et ayant le même nom que la classe avec éventuellement un fichier Test, un fichier Utils et un fichier Error,
- les classes ont une majuscule au début de chaque mot, de même pour les méthodes avec une minuscule au début, et que des minuscule pour les attributs,
- marquer les types et commenter les différentes méthodes et fonctions afin de faciliter l'assemblage des différents modules

6.2 Répartition des tâches

Afin de répartir les tâches d'une manière convenant à l'entièreté du groupe, nous avons décidés de faire un plan clair et précis des tâches à réaliser, puis de laisser chacun choisir les tâches leur convenant et d'attribuer les tâches restantes en fonction des disponibilités et de la charge de travail, et enfin il fut décidé que quelqu'un passe vérifier le bon respect des conventions. Suivant ceci, nous obtenons la répartition suivante :

- **Konchev Martin** s'occupant de l'interface console et du fichier main, puis par la suite de la génération et résolution de grille avec des techniques humaines et la création de la banque de grilles pré-générées ;

- **Barré-Guétrot Ilan** fut réparti sur l'historique de grilles, les classes concernant la grille sudoku ainsi que la génération backtrack et par la suite la résolution de grille humaine et l'interface web ;
- **Dolo Abdourahamane-Abdoul** sur la résolution de grille, la résolution de grille backtrack et ensuite de la résolution de grille humaine ainsi que la récupération de difficulté via le WebScrapping ;
- **Briot-Amadiou Kellyan** se chargea du fichier main, de la transformation de grille en fichier et de l'interaction avec celle-ci, de la grille et des différentes transformations, rotations et inversions de grille par la suite.

7 Bilan et Conclusions

7.1 Fonctionnalités réalisées

7.1.1 Banque de grilles pré-générées

Pour éviter les problèmes de génération lente pendant la soutenance, nous avons décidé d'utiliser un système de banque de grilles pré-générées. L'idée est de ne pas générer les Sudokus en direct devant le jury, car certaines difficultés comme DIFFICILE, EXTREME ou GODMODE peuvent demander plusieurs redémarrages (restarts) et beaucoup de temps avant de trouver une grille valide. Le principe consiste à générer plusieurs grilles à l'avance pour chaque niveau de difficulté, puis à les stocker dans un fichier de banque. Chaque grille sauvegardée contient la grille de jeu, la grille solution complète, la difficulté correspondante, ainsi que les statistiques de résolution obtenues avec SolverHumanStats : comme le nombre d'étapes, les techniques utilisées, la technique maximale atteinte et si la grille est bloquée ou non.

Par exemple, nous pouvons préparer plusieurs grilles pour chaque difficulté : FACILE, MOYEN, DIFFICILE, EXTREME et GODMODE. Ainsi, lorsque l'utilisateur choisit une difficulté dans l'interface console, le programme ne lance plus la génération complète. Il charge directement une grille déjà validée depuis la banque.

Cette méthode présente plusieurs avantages. Elle rend l'exécution beaucoup plus rapide, évite les longues attentes liées aux restarts, supprime le risque qu'une mauvaise grille soit générée pendant la démonstration, et garantit que chaque grille affichée correspond bien à la difficulté demandée. Cela rend la soutenance plus fluide, plus stable et plus professionnelle.

Le générateur de grilles reste bien sûr conservé dans le projet afin de montrer le fonctionnement complet du système. Cependant, pour la démonstration finale, l'utilisation d'une banque de grilles validées permet d'assurer une meilleure fiabilité.

7.2 Comparaison avec cahier des charges

7.3 Limites du projet

7.4 Perspectives

Cependant, le projet pourrait être cible d'encore multiples améliorations, ajouts et optimisations, tels que l'ajout et l'optimisation de techniques de résolution, l'optimisation de la génération de grilles et même l'amélioration de l'interface.

7.4.1 Amélioration des algorithmes

En effet, nous ne faisons qu'écarter la surface de ce qu'est l'iceberg des techniques de résolution, un bon exemple de cela est le site Sudoku Coach [1] qui regorge de ces techniques, de la plus simple comme le singleton nu à la plus complexe tel que méduse 3D.

De plus, les techniques actuellement implémentées ne sont pas forcément optimales et certaines mesures à prendre pour les optimiser serait sûrement possible, tel que la méthode de Grille renvoyant les régions que nous avons ajouté.

7.4.2 Simplification de la génération de grille

Comme énoncé précédemment, la génération de grille est grandement optimisable, nous avons discutés d’une tactique astucieuse à ce sujet : la transformation de grille. Cela consiste à avoir une banque de grilles triées par difficulté, puis, lorsque nous souhaitons générer une grille, de saisir une grille de cette banque de difficulté équivalente et de la passer à travers une fonction de transformation qui peut aléatoirement faire :

- une rotation d’un certain angle (ou symétrie centrale),
- une symétrie axiale,
- et une permutation des valeurs.

Cela permettra de réduire considérablement le temps de génération des grilles.

7.4.3 Interface Web

Les plus grands défauts de l’interface console sont ses interactions limitées et non intuitives ainsi que l’affichage restreint, une bonne solution à ces problèmes serait une interface web, qui marie simplicité et visuel.

Cependant il nous faudrait pour cela un fichier main alternatif sur lequel tourne un serveur afin de pouvoir communiquer entre les modules du sudoku et l’interface web.

7.4.4 Parser

En prenant du recul, cette classe aurait pu être encodée totalement différemment, notamment car il existe déjà des bibliothèques qui facilitent l’écriture et la lecture de données encodées dans des fichiers, mais également qui facilitent le transtypage (ici je pense à la bibliothèque JSON). Ce qui remets également en cause la question de l’existence de la classe FileInteraction, classe qui aurait pu être faite différemment ou alors simplement supprimée.

8 Bibliographie

Les ressources utilisées pendant le projet concernent principalement les techniques de résolution de Sudoku, la comparaison des difficultés et la documentation des outils Python.

Références

- [1] **Sudoku Coach**. Ressource la plus utilisée dans le projet. Elle a servi à comparer nos grilles avec une estimation externe de difficulté et à vérifier les niveaux obtenus par notre solveur humain.
<https://sudoku.coach/>
- [2] **Documentation officielle de Python**. Utilisée pour les bibliothèques standards comme `random`, `os` et `pickle`.
<https://docs.python.org/3/>
- [3] **Documentation Flask**. Utilisée pour la partie interface web du projet.
<https://flask.palletsprojects.com/>
- [4] **Documentation Git**. Utilisée pour la gestion de version et le travail collaboratif.
<https://gitlab.etu.umontpellier.fr/>
- [5] **Cours d’Algorithmique 4**. Chapitre 2 : Recherche exhaustive et backtrack, L3 Informatique, Université de Montpellier. Utilisé pour la résolution de grilles par algorithme de backtracking.
Support de cours fourni au format PDF : `Cours2-Exhaustif.pdf`. Ouvrir le PDF du cours
- [6] **Cours d’Ateliers de Génie Logiciel**. Cours 4 : End2End Testing et Selenium, L3 Informatique, Université de Montpellier. Utilisé pour le sudoku scraping.
Support de cours fourni au format PDF : `CoursE2ETesting_Selenium.pdf`. Ouvrir le PDF du cours

9 Annexes

Cette section regroupe plusieurs captures d'écran obtenues lors des tests du programme et les fonctionnalités de notre projet. Elles illustrent l'affichage des grilles dans l'interface console, les différents niveaux de difficulté, ainsi que la résolution par méthodes humaines.

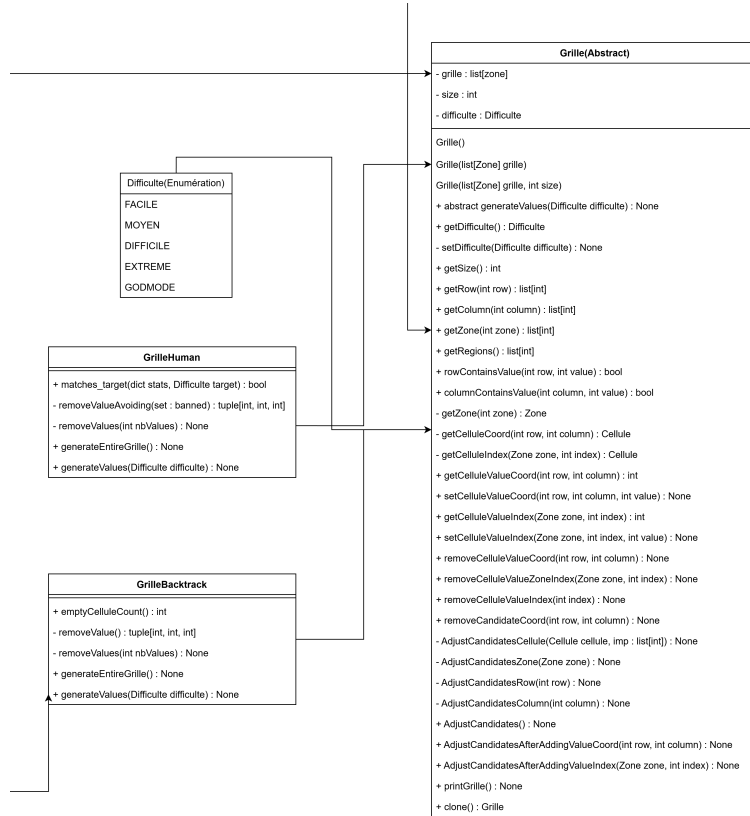


FIGURE 2 – Le diagramme UML représentant les classes des grilles

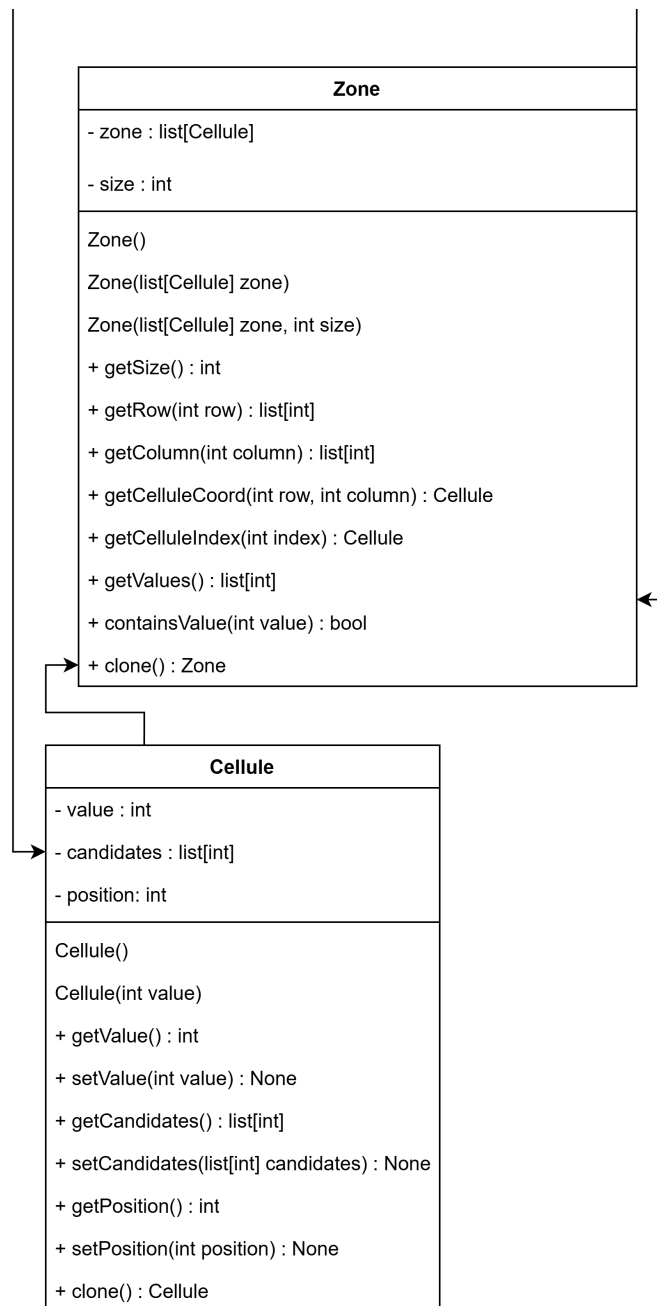


FIGURE 3 – Le diagramme UML représentant la classe Zone et Cellule

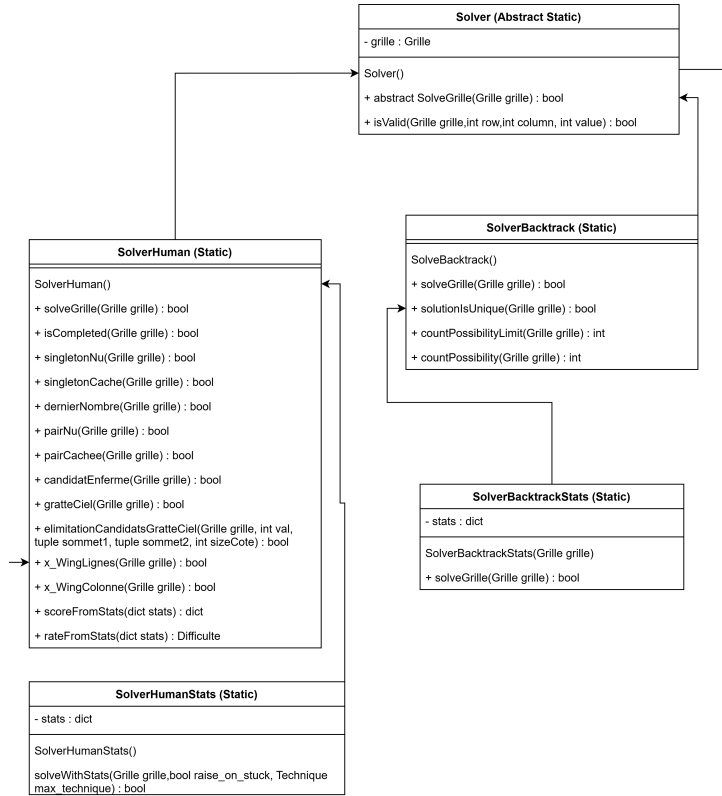


FIGURE 4 – Le diagramme UML représentant les classes de Solver

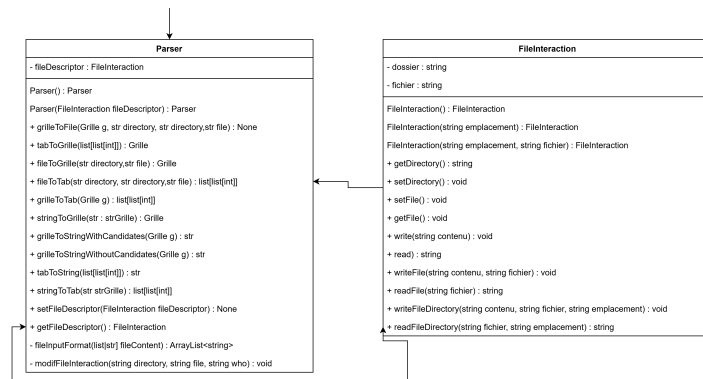


FIGURE 5 – Le diagramme UML représentant la classe Parser

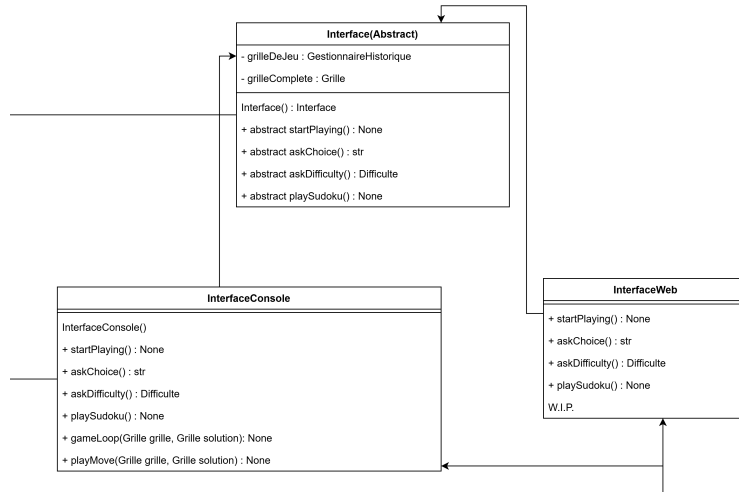


FIGURE 6 – Le diagramme UML représentant les classes d'Interface

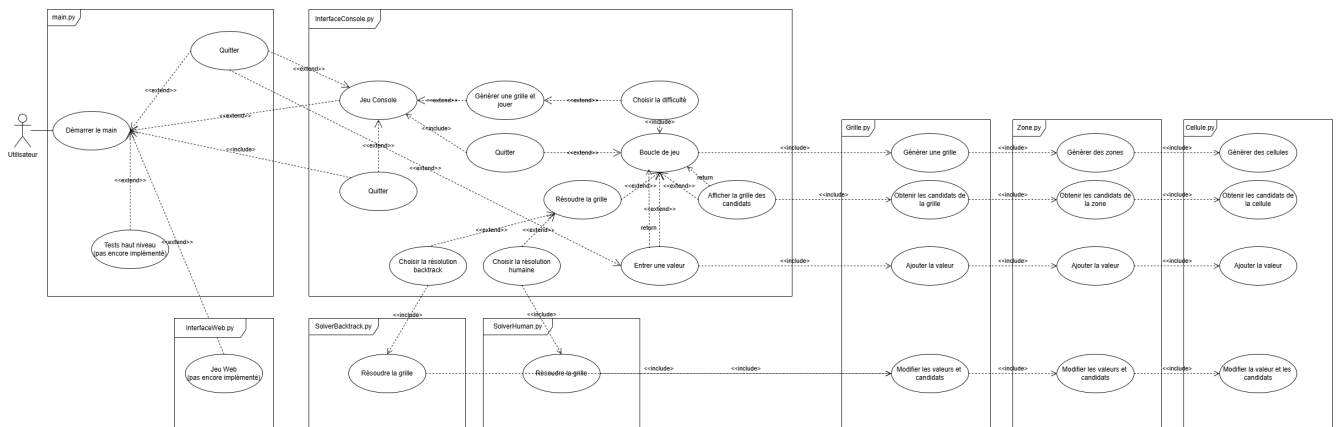


FIGURE 7 – Le diagramme de cas d'utilisation UML

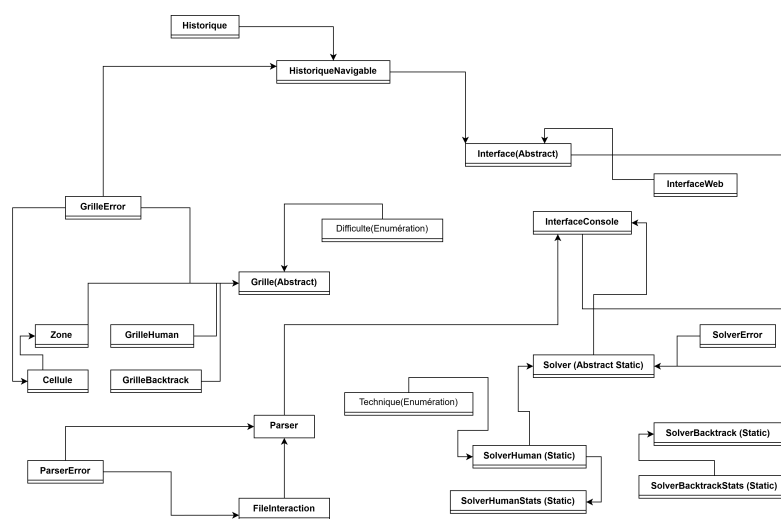


FIGURE 8 – Le diagramme de classes UML entier et simplifié

```

=====
===== Sudoku =====
1 : Jeu dans la console
2 : Serveur web
3 : Tests haut niveau
4 : Sortir
=====
Faites votre choix : 1
=====
===== Bienvenue dans : =====
===== Sudoku Interface Console =====
=====
1. Générer une grille et joueur
2. Quitter
Votre choix: 1

Choisissez une difficulté:
1. Facile
2. Moyen
3. Difficile
4. Extrême
5. God Mode
Votre choix: 1

----- Restart FACILE 1/10 -----

Difficulté estimée (méthodes humaines): Difficulte.FACILE
Steps: 26
Max technique: SINGLETON_NU
Difficulté estimée (sudoku.coach): Facile (score: 1)
. . . | 9 7 . | . 6 3
4 6 . | 2 5 8 | . 9 .
9 . 7 | . 1 3 | 5 8 .
-----
6 8 9 | 4 3 . | 7 . .
7 1 . | 8 . . | 6 3 5
. 3 5 | 1 . 7 | 8 4 .
-----
. 7 2 | 3 . 6 | 9 . 8
3 9 . | 5 2 1 | 4 7 6
5 . . | 7 8 9 | 3 2 1

Erreurs: 0/3

```

FIGURE 9 – Exemple d’une grille facile affichée dans l’interface console

```

=====
===== Bienvenue dans : =====
===== Sudoku Interface Console =====
=====
1. Générer une grille et joueur
2. Quitter
Votre choix: 1

Choisissez une difficulté:
1. Facile
2. Moyen
3. Difficile
4. Extrême
5. God Mode
Votre choix: 3

----- Restart DIFFICILE 1/15 -----

----- Restart DIFFICILE 2/15 -----

----- Restart DIFFICILE 3/15 -----
paire cachée appliquée : [3, 7] dans cellules 2 et 20
paire cachée appliquée : [8, 9] dans cellules 27 et 36

Difficulté estimée (méthodes humaines): Difficulte.DIFFICILE
Steps: 53
Max technique: PAIR_CACHEE
Difficulté estimée (sudoku.coach): Modérément difficile (score: 4)

. . . | . . . | . . 6
. 5 9 | . . . | 7 . .
. 8 . | . . . | . 4 5
-----
. . . | 1 3 . | 6 . 4
. 3 . | . 6 . | . . .
. . . | 8 . 2 | 9 . .
-----
7 9 . | 6 8 3 | 5 . 1
. 1 8 | . 2 . | . 6 .
. . 6 | . . 1 | . . 3

Erreurs: 0/3

```

FIGURE 10 – Exemple d'une grille difficile affichée dans l'interface console

```

Actions disponibles:
1. Entrer une valeur
2. Afficher la grille des candidats
3. Résoudre automatiquement la grille (backtrack)
4. Résoudre automatiquement la grille (méthode humaine)
5. Quitter
Votre choix: 4

Résolution avec méthode humaine...
paire cachée appliquée : [5, 8] dans cellules 38 et 42
gratte ciel appliqué

Résolution humaine terminée.
Solved: True | Steps: 55 | MaxTech: 7
Counts: {'SINGLETON_NU': 28, 'DERNIER_NOMBRE': 21, 'SINGLETON_CACHE': 2, 'PAIR_NU': 0, 'PAIR_CACHEE': 1, 'CANDIDAT_ENFERME': 2, 'GRATTE_CIEL': 1}
6 4 5 | 9 1 3 | 2 7 8
7 2 3 | 5 6 8 | 9 4 1
8 1 9 | 7 2 4 | 6 5 3
-----
5 9 2 | 6 7 1 | 8 3 4
1 7 8 | 4 3 9 | 5 2 6
4 3 6 | 8 5 2 | 1 9 7
-----
3 8 1 | 2 9 7 | 4 6 5
2 5 7 | 1 4 6 | 3 8 9
9 6 4 | 3 8 5 | 7 1 2

```

FIGURE 11 – Exemple d'une grille extrême résolue avec des techniques humaines dans l'interface console

```

Voulez-vous voir les étapes ?
1. Oui
2. Non
Votre choix: 1
Etape 1 - DERNIER_NOMBRE : valeur 8 en ligne 5, colonne 4
Etape 2 - DERNIER_NOMBRE : valeur 6 en ligne 4, colonne 5
Etape 3 - DERNIER_NOMBRE : valeur 3 en ligne 6, colonne 9
Etape 4 - SINGLETON_NU : valeur 3 en ligne 2, colonne 4
Etape 5 - SINGLETON_NU : valeur 7 en ligne 3, colonne 4
Etape 6 - SINGLETON_NU : valeur 7 en ligne 4, colonne 2
Etape 7 - DERNIER_NOMBRE : valeur 3 en ligne 4, colonne 3
Etape 8 - SINGLETON_NU : valeur 6 en ligne 7, colonne 4
Etape 9 - DERNIER_NOMBRE : valeur 4 en ligne 8, colonne 4
Etape 10 - SINGLETON_NU : valeur 7 en ligne 7, colonne 3
Etape 11 - SINGLETON_NU : valeur 8 en ligne 7, colonne 9
Etape 12 - SINGLETON_CACHE : valeur 9 en ligne 1, colonne 3
Etape 13 - SINGLETON_NU : valeur 5 en ligne 3, colonne 3
Etape 14 - DERNIER_NOMBRE : valeur 6 en ligne 9, colonne 3
Etape 15 - SINGLETON_CACHE : valeur 6 en ligne 1, colonne 1
Etape 16 - SINGLETON_CACHE : valeur 9 en ligne 3, colonne 7
Etape 17 - SINGLETON_CACHE : valeur 3 en ligne 3, colonne 8
Etape 18 - SINGLETON_NU : valeur 2 en ligne 7, colonne 8
Etape 19 - DERNIER_NOMBRE : valeur 3 en ligne 7, colonne 1
Etape 20 - SINGLETON_NU : valeur 1 en ligne 1, colonne 8
Etape 21 - DERNIER_NOMBRE : valeur 2 en ligne 1, colonne 5
Etape 22 - DERNIER_NOMBRE : valeur 2 en ligne 2, colonne 7
Etape 23 - DERNIER_NOMBRE : valeur 1 en ligne 2, colonne 2
Etape 24 - DERNIER_NOMBRE : valeur 4 en ligne 4, colonne 8
Etape 25 - SINGLETON_NU : valeur 8 en ligne 3, colonne 6
Etape 26 - DERNIER_NOMBRE : valeur 1 en ligne 3, colonne 5
Etape 27 - DERNIER_NOMBRE : valeur 8 en ligne 9, colonne 5
Etape 28 - DERNIER_NOMBRE : valeur 2 en ligne 9, colonne 6
Etape 29 - SINGLETON_NU : valeur 5 en ligne 8, colonne 7
Etape 30 - SINGLETON_NU : valeur 1 en ligne 4, colonne 7
Etape 31 - DERNIER_NOMBRE : valeur 5 en ligne 4, colonne 9
Etape 32 - DERNIER_NOMBRE : valeur 3 en ligne 9, colonne 7
Etape 33 - SINGLETON_NU : valeur 2 en ligne 8, colonne 1
Etape 34 - DERNIER_NOMBRE : valeur 9 en ligne 8, colonne 9
Etape 35 - DERNIER_NOMBRE : valeur 1 en ligne 9, colonne 9
Etape 36 - SINGLETON_NU : valeur 4 en ligne 3, colonne 1
Etape 37 - DERNIER_NOMBRE : valeur 2 en ligne 3, colonne 2
Etape 38 - DERNIER_NOMBRE : valeur 5 en ligne 9, colonne 1
Etape 39 - DERNIER_NOMBRE : valeur 4 en ligne 9, colonne 2

LE JEU EST TERMINÉ !

```

FIGURE 12 – Exemple des étapes d’une grille moyen resolue avec des techniques humaines dans l’interface console

```

===== Bienvenue dans : =====
===== Sudoku Interface Console =====
1. Générer une grille et jouer
2. Quitter
Votre choix: 1

Choisissez une difficulté:
1. Facile
2. Moyen
3. Difficile
4. Extrême
5. God Mode
Votre choix: 5

Grille chargée depuis la banque :
Difficulté demandée : GODMODE
Difficulté estimée (méthodes humaines) : GODMODE
Steps : 22
Max technique : CANDIDAT_ENFERME
Difficulté estimée (sudoku.coach): Bestial (score: 7)
+ . . . | . 7 . | . 1 .
+ . . | 4 2 . | . 8 .
+ 9 . | . . . | 2 . 3
+-----+-----+-----+
+ 8 . | . . . | . 3 .
+ 3 7 | . 9 6 | . . .
+ 5 6 | 7 . . | 4 . .
+-----+-----+-----+
+ 6 . 3 | . . 4 | 8 5 2
+ . 9 | . . . | . . .
+ . . | 5 . 2 | . . .

Cette grille ne peut pas être résolue uniquement avec les techniques humaines actuellement implémentées dans ce projet (dernier nombre, singleton nu, singleton caché, paire nue, paire cachée, candidat enfermé, gratte-ciel).

Erreurs: 0/3

```

FIGURE 13 – Chargement d’une grille GodMode depuis la banque de grilles pré-générées